integration (CI) pipelines, tools such as SonarQube, Checkmarx, Fortify, or GitHub Advanced Security automatically scan each new commit or pull request, ensuring that insecure code is caught before it moves further down the release pipeline.

The benefits of SAST go beyond just catching bugs—it reduces the cost and effort of fixing issues, improves overall code quality, and helps maintain compliance with security regulations like OWASP Top 10, PCI DSS, and ISO standards. The earlier a vulnerability is found, the cheaper it is to fix, making SAST not only a security best practice but also a smart investment in long-term development efficiency.

In short, SAST acts as a security microscope in the DevSecOps process: it magnifies the smallest cracks in your code, giving teams the chance to seal them early, strengthen their applications, and release software with confidence.

## Dynamic Application Security Testing (DAST)

Dynamic Application Security Testing, or DAST, is another cornerstone of the DevSecOps toolchain, but unlike SAST—which examines the code without running it—DAST focuses on finding vulnerabilities while the application is running. It's often called a black-box testing method because the tester (or tool) doesn't need to know the internal structure of the code. Instead, it interacts with the application from the outside, just like a real user—or, more importantly, like an attacker would.

DAST works by simulating real-world attacks against a deployed version of the application, typically in a staging or testing environment. It sends crafted requests, inputs, and payloads to the application, then observes the responses to detect signs of security weaknesses. For example, it might enter malicious SQL queries into a login form to check for SQL injection vulnerabilities, attempt to inject JavaScript to test for cross-site scripting (XSS), or try manipulating API endpoints to bypass authentication. Because it tests the application in its operational state, DAST can uncover runtime issues that SAST might miss—such as problems with server configuration, authentication logic, third-party integrations, or how the app handles unexpected inputs.

One of DAST's strengths is that it evaluates the complete application, including its infrastructure, middleware, and runtime environment. This makes it particularly effective for catching vulnerabilities caused by misconfigurations**,** insecure error messages, or runtime dependencies. However, since DAST operates without direct access to the code, it may not pinpoint the exact location of a flaw in the codebase—developers often need to combine DAST results with SAST or manual code review to trace and fix the root cause.

In a DevSecOps workflow, DAST is typically integrated into the continuous delivery (CD) phase, running automated scans on staging builds before they are pushed to production. Popular DAST tools include OWASP ZAP, Burp Suite, Netsparker, and Acunetix. These tools can be configured to run full security sweeps after each deployment, ensuring that new features or changes don't introduce vulnerabilities into the live environment.

The benefits of DAST are significant: it mimics how an external attacker might approach the application, tests real-world execution paths, and validates that security controls work as intended. While SAST helps secure the foundation by analysing code, DAST ensures that the final, running product is resistant to attacks in practice. Together, they provide a more complete picture of application security—SAST finds weaknesses early in the code, and DAST confirms those fixes hold up under real-world conditions.

In short, DAST acts as the security crash test for your software: it slams the running application against simulated attacks, measures the damage, and helps you reinforce the weak points before malicious actors have a chance to exploit them.

## Software composition analysis (SCA)

Software composition analysis, or SCA, is a critical part of the DevSecOps toolchain that focuses on securing the third-party and open source components your application relies on. In today's development environment, very few applications are built entirely from scratch—developers speed up delivery by using libraries, frameworks, and packages from public repositories like npm, PyPI, Maven Central, or GitHub. While this approach accelerates innovation, it also introduces a hidden layer of risk: if any of those components contain known vulnerabilities, your application inherits them.

SCA tools work by scanning your application's codebase, build files, and dependency manifests (such as package.json, pom.xml, requirements.txt, or Gemfile) to create a detailed inventory of all open source and third-party components in use. This software bill of materials (SBOM) is then checked against public vulnerability databases like the National Vulnerability Database (NVD), GitHub Security Advisories, or vendor-specific feeds to identify outdated or insecure versions. SCA can detect issues such as dependencies with known CVEs (common vulnerabilities and exposures), licences that may be incompatible with your project's usage, and components that are no longer maintained—meaning future patches or fixes may never come.

One of SCA's biggest advantages is its ability to catch vulnerabilities even